Syed Huzaifa Ali (23K-0004) | BAI-7A

ASSIGNMENT BETA | QUESTION 2

Section 2 — Controlled fine-tuning experiment

Qwen2.5-1.5B-Instruct was evaluated without adaptation and with LoRA, QLoRA, DoRA and AdaLoRA. A 2,000-example subset of Glaive Function Calling v2 was divided into 1,800 training and 200 held-out test examples (seed 42). All fine-tuning runs used one epoch, learning rate 5×10^{-5}, and target modules q_proj and v_proj. LoRA, QLoRA and DoRA used rank 8; QLoRA used 4-bit NF4 and AdaLoRA used initial rank 12 and target rank 8.

Method	Trainable params	% reported	GPU GiB	Train min	JSON valid	Tool acc.	Full match
Baseline	—	—	—	—	193/200	96.0%	88.5%
LoRA r8	1,089,536	0.0705%	3.85	8.17	200/200	100.0%	88.5%
QLoRA r8	1,089,536	0.1225%	4.75	17.84	200/200	100.0%	88.0%
DoRA r8	1,139,712	0.0738%	3.85	10.44	200/200	100.0%	89.5%
AdaLoRA	1,634,976	0.1058%	3.88	10.48	194/200	96.5%	89.0%

JSON validity = successful json.loads() parsing; tool accuracy = correct function name; full match = correct function plus exact arguments. GPU memory is the reported peak allocation during training. The parameter percentages use the denominator reported by each loaded model configuration.

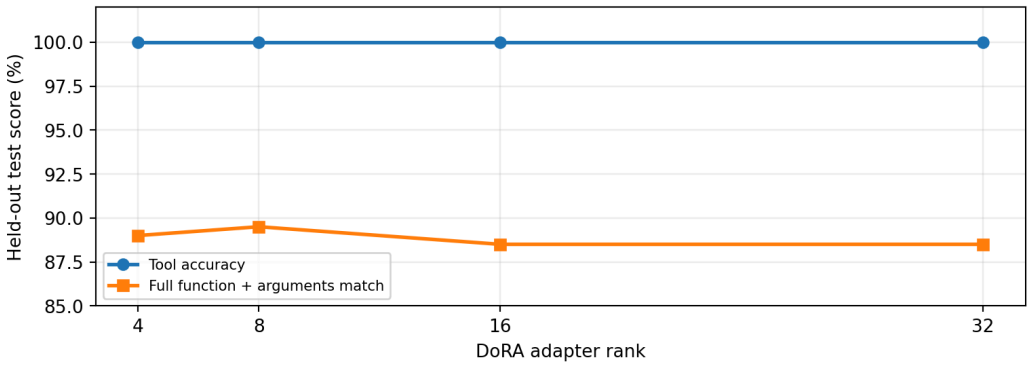
Interpretation

DoRA rank 8 achieved the highest full-match score (179/200, 89.5%), compared with LoRA 177/200, QLoRA 176/200, AdaLoRA 178/200, and baseline 177/200. LoRA trained fastest (8.17 minutes). QLoRA needed 17.84 minutes and had the largest reported peak training memory allocation (4.75 GiB); quantized weight storage does not necessarily imply lower peak training allocation in a specific implementation. The full-match advantage of DoRA over LoRA was just two test cases, making LoRA a reasonable efficiency-oriented alternative.

Section 3 — Does a higher rank help?

DoRA had the highest rank-8 full-match score, so the additional rank experiments used DoRA at r = 4, 16, and 32 with the same data and training configuration.

DoRA rank	Trainable params	Peak GPU GiB	Train min	Tool accuracy	Full match
4	594,944	3.844	10.45	100.0%	89.0%
8	1,139,712	3.852	10.44	100.0%	89.5%
16	2,229,248	3.869	10.64	100.0%	88.5%
32	4,408,320	3.901	10.62	100.0%	88.5%



All tested ranks selected the correct function on all 200 examples. Full match peaked at rank 8 (89.5%). Doubling rank from 8 to 16 nearly doubled adapter parameters (1.14M to 2.23M) without improving performance; rank 32 raised that count to 4.41M and still matched only 88.5%. Rank 8 is the observed accuracy-oriented choice, while rank 4 offers almost the same score with fewer trainable parameters.

Section 4 — Production decision and failure analysis

1. Which method would I ship?

For this measured function-calling prototype, I would choose DoRA rank 8 because it achieved 100% JSON validity, 100% function-name accuracy, and the best full match (89.5%). Its training allocation was about 3.85 GiB and training took 10.44 minutes. LoRA rank 8 is a simpler and faster alternative, with 88.5% full match after 8.17 minutes. The results do not include measured inference latency or throughput, so I would benchmark serving speed before making a performance claim. DoRA magnitude/direction updates can be merged into full-precision weights for deployment; QLoRA quantization requires a compatible quantized-serving or dequantization approach.

2. Five errors from the best model (DoRA r8)

1. Customer: Latest news headlines
Expected: {"arguments": {}, "name": "get_news_headlines"}
Actual: {"arguments": {"name": "get_news_headlines"}, "name": "get_news_headlines"}
Error: Extra argument

2. Customer: Random fact
Expected: {"arguments": {}, "name": "generate_random_fact"}
Actual: {"arguments": {"name": "generate_random_fact"}, "name": "generate_random_fact"}
Error: Extra argument

3. Customer: Feeling down; share something inspiring
Expected: {"arguments": {}, "name": "generate_random_quote"}
Actual: {"arguments": {"functions": ["generate_random_quote"]}, "name": "generate_random_quote"}
Error: Extra argument

4. Customer: Create a todo: Grocery Shopping; Buy groceries for the week; due 2022-03-15
Expected: {"arguments": {"description": "Buy groceries for the week", "due_date": "2022-03-15", "title": "Grocery Shopping"}, "name": "create_todo"}
Actual: {"arguments": {"description": "Buy groceries for the week", "title": "Grocery Shopping"}, "name": "create_todo"}
Error: Missing required argument

5. Customer: Interested in gaming laptops
Expected: {"arguments": {"category": "electronics", "query": "gaming laptop"}, "name": "search_product"}
Actual: {"arguments": {"category": "gaming", "query": "laptops"}, "name": "search_product"}
Error: Incorrect argument values

3. Error pattern

All five failures used the correct function name and valid JSON but supplied incorrect arguments. Three inserted unnecessary fields, one omitted a due date, and one misclassified a product-search category/query. Thus, argument precision and schema compliance are the dominant observed errors. A production backend should validate permitted fields and required arguments, and independently authorize state-changing calls such as refunds.

4. Multi-agent adapter design

A router, refund agent, and shipping agent can share one frozen base model while loading separate lightweight adapters for different behaviors. This avoids training and storing three full base models. A key risk during live adapter switching is that concurrent requests might execute under the wrong adapter. Pin the adapter identity per request, or use isolated serving workers, and keep tool permissions independent of the language model.

5. What surprised me?

The untuned baseline already achieved 177/200 exact matches, and the best adapted model improved this by only two. Increasing DoRA rank from 8 to 16 or 32 did not help exact matching even though trainable parameters rose sharply. AdaLoRA also produced the same raw predictions as the baseline for 191/200 examples, indicating little observed behavioral change in this setup. Finally, QLoRA used more allocated training memory and ran longer than LoRA despite storing its frozen weights in 4-bit form.